دانشگاه صنعتی خواجه نصیرالدین طوسی

دانشکدهٔ مهندسی برق - گروه رباتیک و هوش مصنوعی

پروژه نهایی

درس هوش مصنوعی

موضوع : تشخیص علائم راهنمایی رانندگی با روش های CNN و SVM

استاد درس: دکتر سعید خان کلانتری

نام و نام خانوادگی	محمد امین حری فراهانی
شمارهٔ دانشجویی	۴۰۲۱۶۶۷۳
تاریخ	بهمن ماه ۱۴۰۴

فهرست مطالب

۱ پیاده‌سازی با شبکه CNN ۳

۲ شبکه عصبی کانولوشنی چیست ؟ ۳

۳ تحلیل ساختار شبکه ۳

۴ پروژه ما ۴

۵ دیتاست ۴

۱.۵ معرفی دیتاست ۴

۲.۵ ساختار دیتاست مورد استفاده ۴

۶ پیش‌پردازش و آماده‌سازی داده ۴

۱.۶ بلوک‌های کانولوشن ۶

۷ فرآیند آموزش ۸

۸ نتایج ۹

۹ ماتریس درهم‌ریختگی ۱۰

۱.۹ کلاس‌هایی با بیشترین خطا ۱۲

۲.۹ بررسی Classification Report ۱۲

۱۰ نام کلاس‌های دیتاست ۱۴

۱۱ آزمایش و تست مدل برای عکس های واقعی ۱۵

۱۲ پیاده سازی به روش HOG + SVM ۱۶

۱۳ SVM چیست ؟ ۱۶

۱۴ HOG چیست ؟ ۱۷

۱۵ استخراج ویژگی‌ها با HOG ۱۸

۱.۱۵ مرحله ۴: استخراج هیستوگرام رنگ ۱۹

۲.۱۵ ساخت و آموزش مدل ۲۰

۳.۱۵ مرحله ۴: ماتریس درهم‌ریختگی (Confusion Matrix) ۲۲

۱۶ مقایسه روش های SVM و CNN ۲۲

۱۷ مقایسه دقت کلی ۲۲

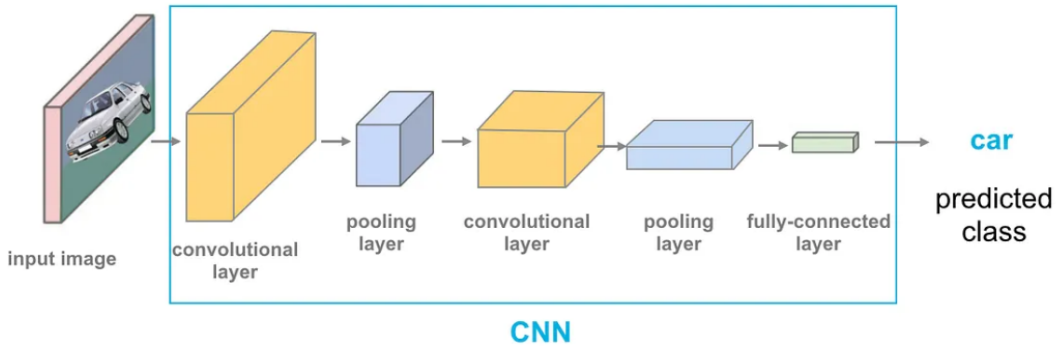
۱۸ مخزن گیت هاب ۲۳

۱ پیاده‌سازی با شبکه CNN

۲ شبکه عصبی کانولوشنی چیست ؟

شبکه‌های عصبی کانولوشنی (CNN) دسته‌ای از شبکه‌های عصبی عمیق هستند که به‌طور خاص برای پردازش داده‌های تصویری و دوبعدی طراحی شده‌اند. در این شبکه لایه‌های کانولوشن با اعمال فیلترهای قابل یادگیری بر روی تصویر ورودی، ویژگی‌های مانند لبه‌ها، گوشه‌ها و الگوهای پایه را استخراج می‌کنند. هر فیلتر با حرکت روی تصویر، یک نگاشت ویژگی تولید می‌کند که نشان‌دهنده وجود یک الگوی خاص در نواحی مختلف تصویر است. پس از کانولوشن، معمولاً از توابع فعال‌ساز استفاده می‌شود تا توانایی مدل در یادگیری روابط پیچیده افزایش یابد.

به‌منظور کاهش ابعاد داده و افزایش پایداری شبکه نسبت به تغییرات جزئی، از لایه‌های Pooling استفاده می‌شود. این لایه‌ها با کاهش وضوح نگاشت‌های ویژگی، علاوه بر کاهش پیچیدگی محاسباتی، از بیش‌برازش جلوگیری می‌کنند. در نهایت، در لایه خروجی شبکه، بسته به نوع مسئله، از توابعی مانند Softmax یا Sigmoid استفاده می‌شود تا خروجی شبکه به‌صورت احتمال یا برچسب نهایی تولید گردد.



۳ تحلیل ساختار شبکه

اولین بلوک زردرنگ نشان‌دهنده لایه کانولوشن است که با اعمال فیلترهای قابل یادگیری روی تصویر، ویژگی‌های پایه‌ای مانند لبه‌ها و الگوهای ماشین را استخراج می‌کند.

در ادامه، بلوک آبی‌رنگ لایه Pooling را نشان می‌دهد که وظیفه آن کاهش ابعاد فضایی داده‌ها و حفظ ویژگی‌های مهم است.

سپس یک لایه کانولوشن دیگر قرار دارد که ویژگی‌های استخراج‌شده در مراحل قبلی را ترکیب کرده و الگوهای پیچیده‌تر را شناسایی می‌کند.

پس از آن، یک لایه Pooling دوم برای فشرده‌سازی بیشتر اطلاعات به‌کار رفته است.

در مرحله پایانی، خروجی لایه‌های کانولوشن و Pooling به لایه تمام‌متصل (Fully-Connected) منتقل می‌شود که نقش تصمیم‌گیری نهایی را بر عهده دارد.

۴ پروژه ما

هدف این پروژه توسعه یک مدل طبقه‌بندی تصویر است که قادر باشد نوع یک علامت جاده‌ای را از تصویر ورودی تشخیص دهد. در این پروژه تلاش شد مدلی پیاده‌سازی شود که هم قابل تحلیل باشد و هم عملکرد خوبی داشته باشد.

۵ دیتاست

۱.۵ معرفی دیتاست

دیتاست مورد استفاده در این پروژه **Benchmark Recognition Sign Traffic German (GTSRB)** است که شامل تصاویر رنگی از علائم راهنمایی و رانندگی آلمان می‌باشد. لینک دیتاست :

Kaggle

۲.۵ ساختار دیتاست مورد استفاده

ساختار پوشه ای :

- `Train/0/`، `Train/1/`، ...، `Train/42/` — پوشه‌های کلاس‌های آموزشی
- `Test/` — تصاویر تست
- نام کلاس های دیتاست : ۱۰

۶ پیش‌پردازش و آماده‌سازی داده

مراحل پیش‌پردازش شامل موارد زیر است:

- تبدیل فضای رنگی BGR (OpenCV) به RGB
- Resize تصاویر به اندازه 30×30 پیکسل (برای سرعت و هماهنگی با معماری مدل)
- نرمال‌سازی پیکسل‌ها
- تقسیم مجموعه به آموزش و اعتبارسنجی (حدود ۲۰٪ اعتبارسنجی)

ابتدا تصاویر از فضای رنگی BGR به RGB تبدیل می‌شوند (cv2.imread باعث میشود که OpenCV بصورت BGR تبدیل کند و ما باید آن را به RGB بازگردانیم چون ورودی شبکه شما تصاویر RGB هست) . سپس اندازه تصاویر به 30×30 پیکسل تغییر داده می‌شود تا ورودی‌ها دارای ابعاد یکسان بوده و هزینه محاسباتی کاهش یابد. در ادامه، مقادیر پیکسلی با تقسیم بر ۲۵۵ نرمال‌سازی می‌شوند تا فرآیند یادگیری پایدارتر و سریع‌تر انجام شود.

```
NUM_CATEGORIES = 43
IMG_HEIGHT = 30
IMG_WIDTH = 30

image_data = []
image_labels = []

for i in range(NUM_CATEGORIES):
    path = os.path.join(train_path, str(i))
    images = os.listdir(path)
    for img_name in images:
        try:
            img_path = os.path.join(path, img_name)
            image = cv2.imread(img_path)
            image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
            image = cv2.resize(image, (IMG_WIDTH, IMG_HEIGHT))
            image_data.append(image)
            image_labels.append(i)
        except Exception as e:
            print(f"Error loading image {img_name}: {e}")

 train_test_split(X, y, test_size=0.2, random_state=42, shuffle=True)
```

با استفاده از یک حلقه تکرار، وارد تک‌تک ۴۳ پوشه دیتاست می شویم و تصاویر فراخوانی می‌شوند:

- **فراخوانی:** استفاده از تابع `cv2.imread` برای تبدیل فایل تصویر به ماتریس‌های عددی.
- **اصلاح فضای رنگی:** به دلیل اینکه OpenCV تصاویر را به صورت BGR بارگذاری می‌کند، با تابع `cv2.cvtColor` تصاویر به فضای رنگی **RGB** منتقل می‌شوند.
- **تغییر ابعاد :(Resizing)** تمام تصاویر برای یکسان‌سازی ورودی شبکه به ابعاد 30×30 پیکسل تغییر اندازه می‌یابند.
- **تقسیم‌بندی :(Splitting)** با استفاده از تابع `train_test_split`، داده‌ها به دو بخش آموزش (۸۰٪) و اعتبار سنجی (۲۰٪) تقسیم می‌شوند.

۱.۶ بلوک‌های کانولوشن

شبکه شامل دو بلوک کانولوشنی متوالی است.

در بلوک اول، از **۳۲ فیلتر کانولوشنی با اندازه‌ی کرنل** 5×5 استفاده شده است. این لایه‌ها وظیفه‌ی استخراج ویژگی‌های سطح پایین تصویر مانند لبه‌ها، گوشه‌ها و الگوهای ساده‌ی رنگی را بر عهده دارند.

در بلوک دوم، تعداد فیلترها به **۶۴ فیلتر با کرنل** 3×3 افزایش یافته است. در این مرحله، شبکه به استخراج ویژگی‌های سطح بالاتر مانند شکل کلی علائم، نمادها و ترکیب رنگ‌ها می‌پردازد. کوچک‌تر شدن اندازه‌ی کرنل به شبکه اجازه می‌دهد جزئیات دقیق‌تری را مدل‌سازی کند.

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPool2D, Dense, Flatten, Dropout

model.add(Conv2D(filters=32, kernel_size=(5,5), activation='relu', input_shape=(30,
    30, CHANNELS)))
.
.
.
model.add(Conv2D(filters=64, kernel_size=(3, 3), activation='relu'))
.
.
.
model.add(Flatten())
model.add(Dense(256, activation='relu'))
model.add(Dropout(rate=0.5))
model.add(Dense(NUM_CATEGORIES, activation='softmax'))
```

- Conv2D: استخراج ویژگی‌های محلی تصویر (لبه‌ها، بافت‌ها)
- Flatten و Dense: تبدیل ویژگی‌ها به یک بردار و تصمیم‌گیری نهایی
- ReLU : این تابع فعال ساز باعث می‌شود جاهایی از عکس که وییگی مورد نظر ما نیست، کاملاً سیاه (بی‌اثر) شوند و فقط بخش‌هایی که دایره در آن‌ها پیدا شده، با اعداد مثبت "درخشان" شوند تا لایه‌های بعدی فقط روی آن‌ها تمرکز کنند.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	2,432
conv2d_1 (Conv2D)	(None, 22, 22, 32)	25,632
max_pooling2d (MaxPooling2D)	(None, 11, 11, 32)	0
dropout (Dropout)	(None, 11, 11, 32)	0
conv2d_2 (Conv2D)	(None, 9, 9, 64)	18,496
conv2d_3 (Conv2D)	(None, 7, 7, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 3, 3, 64)	0
dropout_1 (Dropout)	(None, 3, 3, 64)	0
flatten (Flatten)	(None, 576)	0
dense (Dense)	(None, 256)	147,712
dropout_2 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 43)	11,051

شکل ۱: معماری شبکه عصبی کانولوشنی استفاده‌شده برای تشخیص علائم راهنمایی و ابعاد خروجی و پارامترهای شبکه

خروجی model summary نشان می‌دهد که شبکه شامل دو بلوک کانولوشنی برای استخراج ویژگی و یک بخش تمام‌متصل برای طبقه‌بندی نهایی است

Shape Output : این ستون ابعاد خروجی هر لایه را نشان می‌دهد. عبارت None در ابتدای هر سطر نشان‌دهنده اندازه دسته داده (batch size) است که در زمان اجرا مشخص می‌شود. اعداد بعدی بیانگر ابعاد مکانی تصویر و تعداد کانال‌ها هستند.

Param #: این ستون تعداد پارامترهای قابل یادگیری هر لایه را نمایش می‌دهد. این پارامترها شامل وزن‌ها و بایاس‌ها هستند. لایه‌هایی مانند Conv2D و Dense دارای پارامتر قابل آموزش‌اند، در حالی که لایه‌هایی مانند MaxPooling و Dropout پارامتری برای یادگیری ندارند.

۷ فرآیند آموزش

```
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy
    '])

model.summary()

epochs = 15

history = model.fit(X_train, y_train, batch_size=32, epochs=epochs,
                    validation_data=(X_val, y_val))
```

- X_train , y_train: داده‌ها و برچسب‌های آموزش
- batch_size = 32: تعداد نمونه‌هایی که قبل از هر به‌روزرسانی وزن‌ها پردازش می‌شوند
- validation_data: مجموعه اعتبارسنجی برای ارزیابی عملکرد مدل پس از هر epoch

خروجی این تابع در متغیر history ذخیره می‌شود که شامل مقادیر loss و accuracy برای داده‌های آموزش و اعتبارسنجی در هر epoch است. این مقادیر برای رسم نمودارهای یادگیری و تحلیل رفتار مدل استفاده می‌شوند.

- **تابع هزینه :** از تابع هزینه categorical_crossentropy برای آموزش شبکه استفاده شده است. تابع categorical_crossentropy اختلاف بین توزیع احتمال پیش‌بینی‌شده و برچسب واقعی را اندازه‌گیری کرده و مدل را به سمت بیشینه‌سازی احتمال کلاس صحیح هدایت می‌کند.

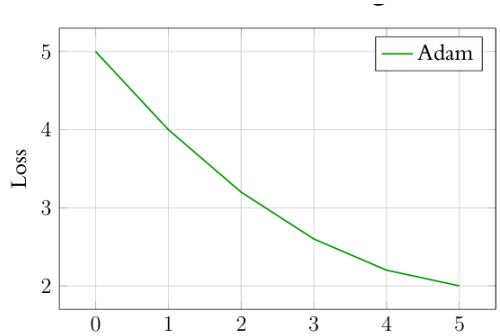
  به دلیل پایداری عددی و همگرایی سریع اکثرا استفاده می شود .

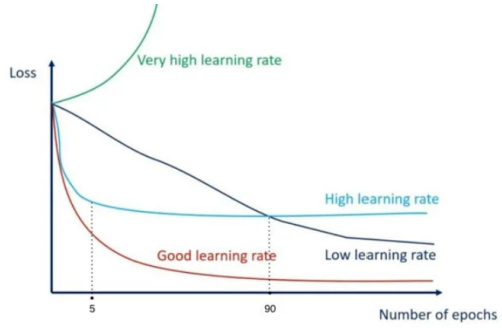
- **معیار ارزیابی :(Metric)** معیار accuracy برای اندازه‌گیری درصد پیش‌بینی‌های صحیح در هر epoch استفاده شده است.

- **epoch :**

  مشخص می‌کند که کل داده‌های آموزشی چند بار به طور کامل به شبکه ارائه شوند. هر epoch شامل یک عبور کامل روی کل مجموعه آموزش است.

- **Optimizer :** الگوریتم Adam برای به‌روزرسانی وزن‌ها انتخاب شده است. این الگوریتم همگرایی سریع و پایداری مناسبی در آموزش شبکه‌های عمیق فراهم می‌کند.



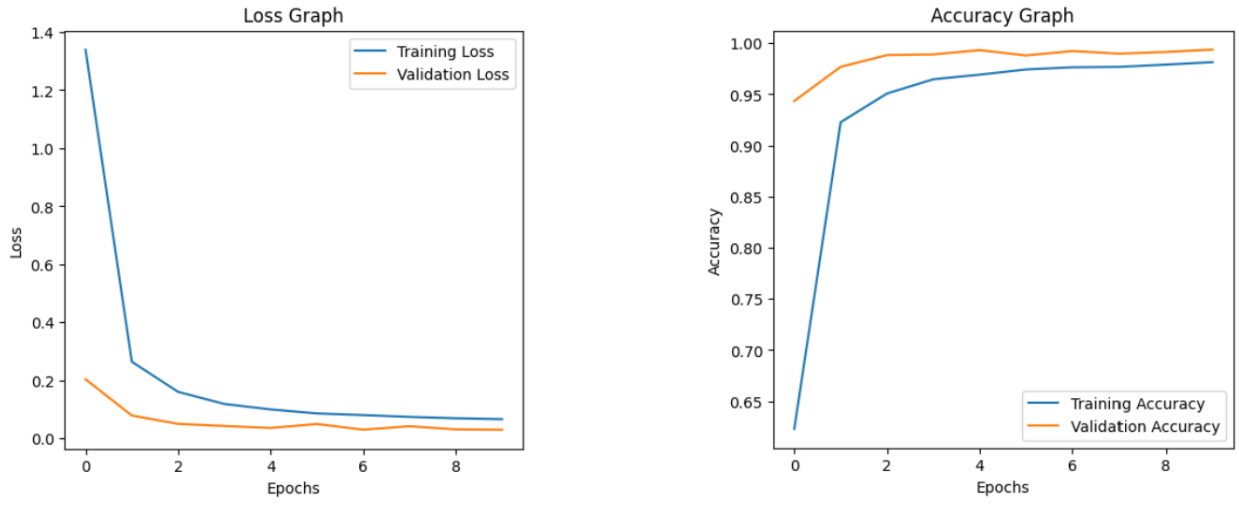


۸ نتایج

```
Epoch 1/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 39s 38ms/step - accuracy: 0.6413 - loss: 1.2662 - val_accuracy: 0.9563 - val_loss: 0.1685
Epoch 2/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 38s 38ms/step - accuracy: 0.9194 - loss: 0.2610 - val_accuracy: 0.9811 - val_loss: 0.0781
Epoch 3/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 37s 38ms/step - accuracy: 0.9476 - loss: 0.1722 - val_accuracy: 0.9855 - val_loss: 0.0569
Epoch 4/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 37s 38ms/step - accuracy: 0.9627 - loss: 0.1255 - val_accuracy: 0.9864 - val_loss: 0.0499
Epoch 5/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 38s 38ms/step - accuracy: 0.9678 - loss: 0.1088 - val_accuracy: 0.9884 - val_loss: 0.0408
Epoch 6/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 37s 37ms/step - accuracy: 0.9721 - loss: 0.0902 - val_accuracy: 0.9929 - val_loss: 0.0240
Epoch 7/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 37s 37ms/step - accuracy: 0.9740 - loss: 0.0845 - val_accuracy: 0.9935 - val_loss: 0.0270
Epoch 8/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 38s 39ms/step - accuracy: 0.9775 - loss: 0.0756 - val_accuracy: 0.9953 - val_loss: 0.0224
Epoch 9/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 37s 37ms/step - accuracy: 0.9797 - loss: 0.0692 - val_accuracy: 0.9921 - val_loss: 0.0309
Epoch 10/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 37s 38ms/step - accuracy: 0.9806 - loss: 0.0645 - val_accuracy: 0.9954 - val_loss: 0.0192
Epoch 11/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 37s 38ms/step - accuracy: 0.9800 - loss: 0.0632 - val_accuracy: 0.9953 - val_loss: 0.0217
Epoch 12/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 37s 38ms/step - accuracy: 0.9819 - loss: 0.0633 - val_accuracy: 0.9948 - val_loss: 0.0226
Epoch 13/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 38s 38ms/step - accuracy: 0.9820 - loss: 0.0599 - val_accuracy: 0.9932 - val_loss: 0.0228
Epoch 14/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 37s 37ms/step - accuracy: 0.9843 - loss: 0.0555 - val_accuracy: 0.9899 - val_loss: 0.0344
Epoch 15/15
981/981 ━━━━━━━━━━━━━━━━━━━━ 37s 37ms/step - accuracy: 0.9852 - loss: 0.0505 - val_accuracy: 0.9946 - val_loss: 0.0212
```

شکل ۲: طول فرآیند

نتایج آموزش مدل CNN : دقت و خطای مدل بر روی داده‌های آموزش و اعتبارسنجی در طول ۱۵ ایپاک مشاهده می‌شود که مدل به سرعت همگرا شده و دقت اعتبارسنجی بالا و پایدار است.



کد نمایش نمودار ها :

```
# Accuracy & Loss plots
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title('Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title('Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')

plt.savefig('accuracy_loss.png', bbox_inches='tight')
```

۹ ماتریس درهم‌ریختگی

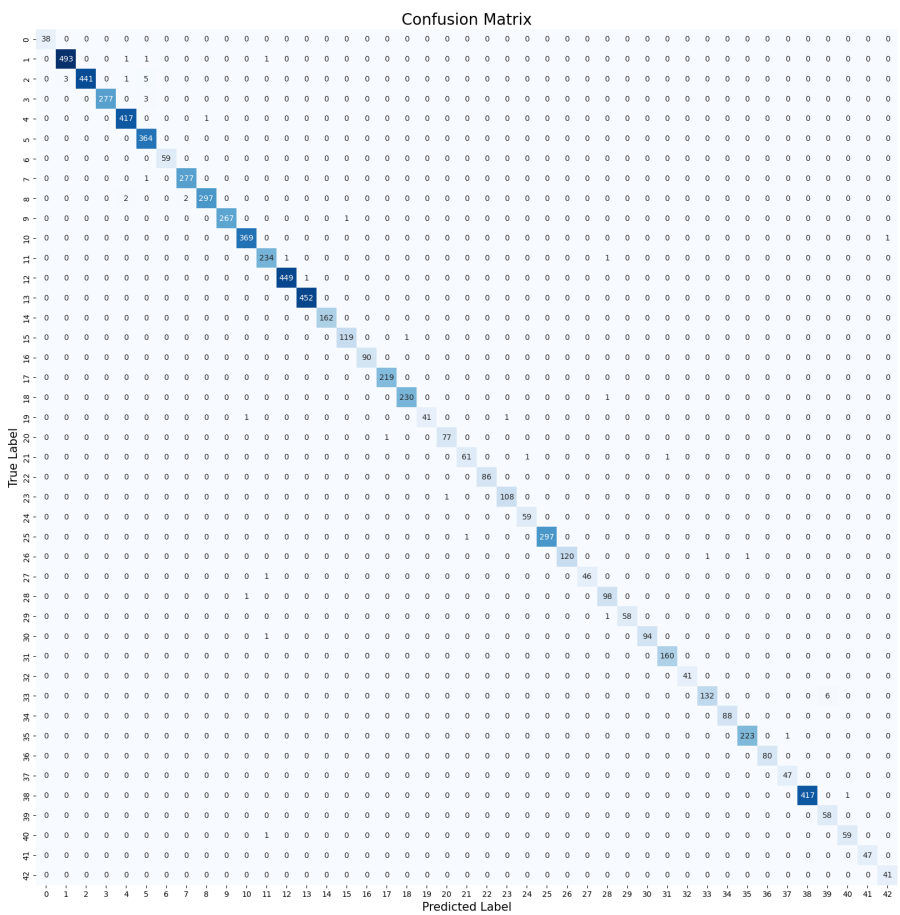
ماتریس درهم‌ریختگی ابزاری استاندارد برای ارزیابی عملکرد مدل‌های طبقه‌بندی است. این ماتریس یک جدول مربعی است که در آن سطرها نشان‌دهنده کلاس‌های واقعی و ستون‌ها نشان‌دهنده کلاس‌های پیش‌بینی‌شده هستند. هر خانهٔ ماتریس تعداد نمونه‌هایی را نشان می‌دهد که به

ترکیب خاصی از برچسب واقعی و پیش‌بینی شده تعلق دارند.

تحلیل ماتریس :

- اگر عناصر روی قطر اصلی ماتریس بزرگ باشند، مدل عملکرد خوبی دارد.
- اگر عناصر خارج از قطر اصلی بزرگ باشند، مدل خطاهای زیادی دارد و نیاز به بهبود دارد.

```
from sklearn.metrics import confusion_matrix, classification_report
pred_probabilities = model.predict(X_val)
pred_classes = np.argmax(pred_probabilities, axis=1)
true_classes = np.argmax(y_val, axis=1)

cm = confusion_matrix(true_classes, pred_classes)

plt.figure(figsize=(20,20))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.savefig('confusion_matrix.png', bbox_inches='tight')
print(classification_report(true_classes, pred_classes))
```



بررسی ماتریس درهم‌ریختگی

ماتریس درهم‌ریختگی نشان می‌دهد که تقریباً تمام مقادیر بزرگ روی قطر اصلی قرار دارند؛ این موضوع بیانگر آن است که مدل در اغلب کلاس‌ها پیش‌بینی صحیح انجام داده است. مقادیر خارج از قطر اصلی بسیار کوچک هستند که نشان‌دهنده نرخ خطای بسیار پایین مدل است.

۱.۹ کلاس‌هایی با بیشترین خطا

- برخی کلاس‌های **محدودیت سرعت** (مانند ۳۰، ۵۰، ۶۰ و ۸۰ کیلومتر بر ساعت) به صورت محدود با یکدیگر اشتباه شده‌اند. این موضوع به دلیل شباهت ظاهری زیاد این تابلوها طبیعی است.
- بین کلاس‌های **left curve Dangerous** و **right curve Dangerous** نیز تعداد کمی خطای متقابل مشاهده می‌شود که ناشی از تقارن نسبی شکل این تابلوها است.
- تمرکز مقادیر بالا روی قطر اصلی نشان‌دهنده دقت بسیار بالای مدل است.
- پراکندگی بسیار کم مقادیر خارج از قطر بیانگر تعمیم‌پذیری مناسب شبکه است.
- بیشترین خطاها در کلاس‌هایی رخ داده که شباهت بصری بالایی دارند، نه در کلاس‌های کاملاً متفاوت؛ این نشان می‌دهد مدل الگوهای معنایی صحیح را یاد گرفته است.

class	Precision	Recall	F1-Score	Support
0	1.00	1.00	1.00	38
1	0.99	0.99	0.99	496
2	1.00	0.98	0.99	450
3	1.00	0.99	0.99	280
4	0.99	1.00	0.99	418
5	0.97	1.00	0.99	364
6	1.00	1.00	1.00	59
⋮	⋮	⋮	⋮	⋮
42	0.98	1.00	0.99	41
Accuracy		0.99		
Macro avg		0.99		
Weighted avg		0.99		

۲.۹ بررسی Classification Report

برای ارزیابی عملکرد مدل، از معیارهای استاندارد طبقه‌بندی چندکلاسه استفاده شده است. هر یک از این معیارها جنبه متفاوتی از عملکرد شبکه را اندازه‌گیری می‌کنند:

- **Precision (دقت پیش‌بینی):** نشان می‌دهد از بین نمونه‌هایی که مدل به عنوان یک کلاس خاص پیش‌بینی کرده است، چه نسبتی واقعاً متعلق به همان کلاس بوده‌اند. بالا بیانگر آن است که مدل به ندرت نمونه‌ای را به اشتباه به یک کلاس نسبت داده است.

- **Recall (حساسیت):** Recall نشان می‌دهد از بین تمام نمونه‌های واقعی یک کلاس، چه نسبتی توسط مدل به درستی شناسایی شده‌اند.
- **F۱-Score:** این معیار میانگین هارمونیک Precision و Recall است و زمانی اهمیت دارد که بخواهیم تعادل بین این دو را بررسی کنیم. نزدیکی F۱-Score به 1.00 نشان می‌دهد مدل هم در جلوگیری از پیش‌بینی‌های اشتباه و هم در شناسایی صحیح نمونه‌ها عملکرد بسیار متعادلی دارد.

در مجموع، نتایج نشان می‌دهد مدل CNN توانسته دقت بالا و عملکردی پایدار در تمامی ۴۳ کلاس داشته باشد

حال میتوانیم مدل آموزش دیده خود را ذخیره کرده و ار آن استفاده کنیم :

```

model.save("MyCNNModel") % for save myModel
print("Model Saved!")

%-------------------------------------%

from tensorflow.keras.models import load_model
import numpy as np
import cv2

model = load_model("MyCNNModel.h5")

def predict_image(image_path):
    img = cv2.imread(image_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (30, 30))
    img = img.astype("float32") / 255.0
    img = np.expand_dims(img, axis=0)

    prediction = model.predict(img)
    class_id = np.argmax(prediction)
    confidence = np.max(prediction)

    print("Predicted class:", class_id)
    print("Confidence:", confidence)

predict_image("test.png")
```

۱۰ نام کلاس‌های دیتاست

جدول ۱: GTSRB

Code	class	Code	class
0	Speed limit (20km/h)	22	Bumpy road
1	Speed limit (30km/h)	23	Slippery road
2	Speed limit (50km/h)	24	Road narrows on the right
3	Speed limit (60km/h)	25	Road work
4	Speed limit (70km/h)	26	Traffic signals
5	Speed limit (80km/h)	27	Pedestrians
6	End of speed limit (80km/h)	28	Children crossing
7	Speed limit (100km/h)	29	Bicycles crossing
8	Speed limit (120km/h)	30	Beware of ice/snow
9	No passing	31	Wild animals crossing
10	No passing veh over 3.5 t	32	End speed + passing limits
11	Right-of-way at intersection	33	Turn right ahead
12	Priority road	34	Turn left ahead
13	Yield	35	Ahead only
14	Stop	36	Go straight or right
15	No vehicles	37	Go straight or left
16	Veh > 3.5 tons prohibited	38	Keep right
17	No entry	39	Keep left
18	General caution	40	Roundabout mandatory
19	Dangerous curve left	41	End of no passing
20	Dangerous curve right	42	End no passing veh > 3.5 t
21	Double curve		

۱۱ آزمایش و تست مدل برای عکس های واقعی

محدودیت‌ها

مدل ارائه‌شده تنها برای تصاویر از پیش برش‌خوردهٔ تابلوهای راهنمایی طراحی شده است. در تصاویر واقعی که شامل پس‌زمینه پیچیده، تغییرات نور و مقیاس هستند، به دلیل اختلاف دامنه داده‌ها، عملکرد مدل کاهش می‌یابد.

برای تست روی یک تصویر واقعی از تابع زیر استفاده شده است:

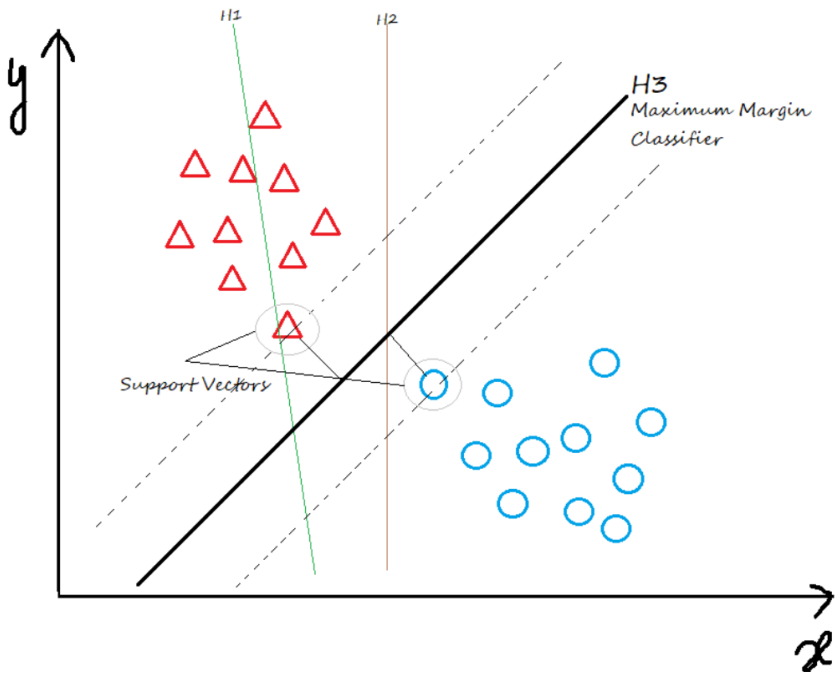
```python
def predict_traffic_sign(image_path):
    img = cv2.imread(image_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_resized = cv2.resize(img, (30,30))
    img_array = np.expand_dims(img_resized/255.0, axis=0)
    pred = model.predict(img_array)
    class_index = np.argmax(pred)
    prob = np.max(pred)
    plt.imshow(img)
    plt.title(f"Prediction: {classes[class_index]} - {prob*100:.2f}%")
    plt.axis('off')
    plt.show()

predict_traffic_sign("test.png")
```

نمونه خروجی : تصویر ورودی همراه با کد کلاس پیش‌بینی‌شده و میزان اطمینان (Confidence)

۱۲ پیاده سازی به روش HOG + SVM

۱۳ SVM چیست ؟

ماشین بردار پشتیبان (Support Vector Machine) یکی از الگوریتم‌های یادگیری نظارت‌شده در حوزه طبقه‌بندی و رگرسیون است که هدف آن یافتن یک مرز تصمیم بهینه برای جداسازی کلاس‌ها می‌باشد. ایده اصلی این روش بر مبنای بیشینه‌سازی فاصله بین داده‌های متعلق به کلاس‌های مختلف و مرز تصمیم است. نمونه‌هایی که دقیقاً در نزدیکی مرز تصمیم قرار دارند و تعیین‌کننده موقعیت هستند، **بردارهای پشتیبان** (Support Vectors) نامیده می‌شوند.



ورودی اولیه مدل یک تصویر رنگی از تابلو راهنمایی است. این به صورت یک آرایه عددی سه‌بعدی ذخیره می‌شود. به عنوان مثال، یک تصویر با ابعاد $30 \times 30 \times 3$

هر پیکسل تصویر شامل سه عدد است که شدت رنگ‌های RGB را نشان می‌دهند. این اعداد پس از نرمال‌سازی در بازه $[0, 1]$ قرار می‌گیرند.

از آنجا که مدل SVM نمی‌تواند مستقیماً با داده‌های سه‌بعدی کار کند، تصویر باید به یک بردار یک‌بعدی تبدیل شود. (Flatten) . در نتیجه، تصویر با ابعاد $30 \times 30 \times 3$ به یک بردار با طول ۲۷۰۰ تبدیل می‌شود:

$$30 \times 30 \times 3 = 2700$$

بنابراین، ورودی یک بردار عددی ۲۷۰۰ بعدی است که نمایانگر شدت رنگ تمام پیکسل‌های تصویر می‌باشد.

پس از این مرحله، ویژگی‌ها با استفاده از روش استانداردسازی مقیاس‌بندی می‌شوند. این کار باعث می‌شود مدل در هنگام یافتن مرز تصمیم عملکرد پایدارتری داشته باشد.

۲) نحوه تشخیص مدل چگونه است؟

پس از آماده‌سازی SVM تلاش می‌کند در فضای ۲۷۰۰ بعدی یک مرز تصمیم (Hyperplane) پیدا کند که تصاویر کلاس‌های مختلف را از یکدیگر جدا کند.

می‌توان فضای ۲۷۰۰ بعدی را به صورت یک فضای بسیار بزرگ در نظر گرفت که هر تصویر یک نقطه در آن است. مدل در این فضا به دنبال صفحه‌ای می‌گردد که:

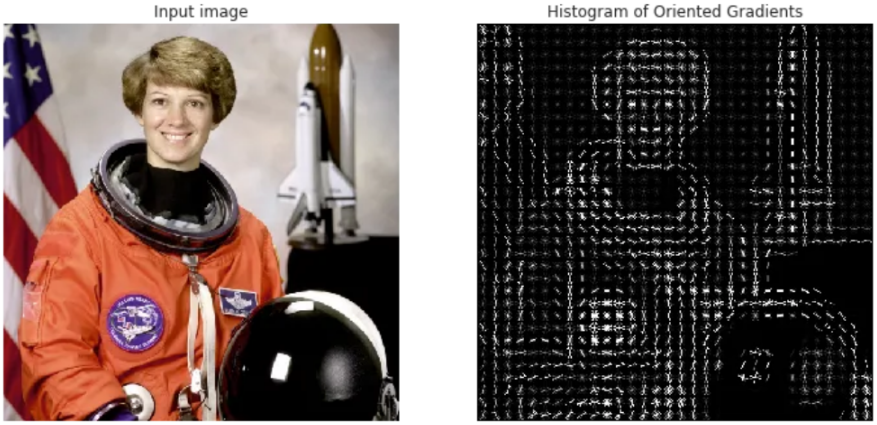
- نقاط متعلق به کلاس‌های مختلف در دو سوی آن قرار گیرند،
- فاصله این صفحه تا نزدیک‌ترین نقاط هر کلاس بیشینه باشد.

این فاصله بیشینه همان **Margin** است و نقاطی که نزدیک‌ترین فاصله را با مرز دارند، **بردارهای پشتیبان** نامیده می‌شوند. این نقاط نقش تعیین‌کننده‌ای در شکل‌گیری مرز تصمیم دارند.

در مرحله پیش‌بینی، یک تصویر جدید دقیقاً همان مراحل پیش‌پردازش (تغییر اندازه، نرمال‌سازی، تخت‌سازی و استانداردسازی) را طی می‌کند. سپس بردار حاصل به مدل داده می‌شود.

مدل با محاسبه موقعیت این بردار نسبت به مرزهای تصمیم، مشخص می‌کند که تصویر به کدام ناحیه از فضای ویژگی‌ها نزدیک‌تر است. در نهایت، خروجی مدل یک برچسب عددی بین ۰ تا ۴۲ است که نمایانگر کلاس پیش‌بینی‌شده تابلو می‌باشد.

۱۴ HOG چیست ؟



Histogram of Oriented Gradients (HOG) یکی از روش‌های استخراج ویژگی در بینایی ماشین است که برای توصیف شکل و ساختار لبه‌های موجود در تصویر به کار می‌رود. ایده اصلی این روش بر مبنای تحلیل گرادیان شدت روشنایی تصویر است.

در این روش ابتدا تصویر به سطح خاکستری تبدیل می‌شود و سپس در هر پیکسل، تغییرات شدت روشنایی در راستاهای افقی و عمودی محاسبه می‌شود. این تغییرات شدت، که به آن‌ها گرادیان گفته می‌شود، نشان‌دهنده وجود لبه یا تغییر ناگهانی در تصویر هستند.

در نهایت، تمام این هیستوگرام‌های نرمال‌سازی‌شده به هم متصل شده و یک بردار ویژگی عددی تشکیل می‌دهند که نمایانگر ساختار لبه‌های تصویر است.

مدل SVM برخلاف شبکه‌های عصبی کانولوشنی، قابلیت استخراج خودکار ویژگی از تصویر را ندارد و صرفاً روی بردارهای عددی کار می‌کند. بنابراین لازم است پیش از آموزش، اطلاعات مهم تصویر به صورت فشرده و معنادار استخراج شوند.

۱۵ استخراج ویژگی‌ها با HOG

گفتیم که لازم است هر تصویر ابتدا به یک **بردار ویژگی** (Feature Vector) تبدیل شود. تابع extract_features دقیقاً همین کار را انجام می‌دهد: از هر تصویر، دو نوع ویژگی استخراج کرده و سپس آن‌ها را با هم ترکیب می‌کند.

```
def extract_features(images):
    features = []
    for img in images:
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        hog_features = hog(gray, orientations=9, pixels_per_cell=(8,8),
                           cells_per_block=(2,2), block_norm='L2-Hys', visualize=
    False)
        hist = cv2.calcHist([img], [0,1,2], None, [8,8,8], [0,256,0,256,0,256])
        hist = cv2.normalize(hist, hist).flatten()
        feature_vector = np.hstack([hog_features, hist])
        features.append(feature_vector)
    return np.array(features)

X_features = extract_features(X)
```

در ابتدا یک لیست خالی به نام features ساخته می‌شود. سپس با حلقه for، روی همه تصاویر موجود در ورودی images پیمایش انجام می‌شود:

- هر بار متغیر img یک تصویر کامل (با ابعاد $32 \times 32 \times 3$) را در خود دارد.
- هدف این حلقه این است که برای هر تصویر، یک بردار ویژگی ساخته شود و در لیست features ذخیره گردد.

تبدیل تصویر به سطح خاکستری

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

HOG بر اساس گرادیان شدت روشنایی کار می‌کند، تصویر ابتدا به حالت خاکستری تبدیل می‌شود. در تصویر خاکستری، هر پیکسل فقط یک مقدار شدت روشنایی دارد، بنابراین محاسبه گرادیان‌ها ساده‌تر و پایدارتر می‌شود.

استخراج ویژگی‌های HOG

```
hog_features = hog(gray, orientations=9, pixels_per_cell=(8,8),
                   cells_per_block=(2,2), block_norm='L2-Hys', visualize=False)
```

در این مرحله از روش HOG برای استخراج ویژگی‌های لبه و ساختار شکل تابلو استفاده می‌شود. روند کلی HOG به صورت زیر است:

۱. **محاسبه گرادیان:** برای هر پیکسل، تغییرات شدت روشنایی در راستای افقی و عمودی محاسبه می‌شود. این تغییرات نشان‌دهنده وجود لبه‌ها است.

۲. **تقسیم تصویر به سلول‌ها (Cell):** تصویر به نواحی کوچک 8×8 پیکسلی تقسیم می‌شود (pixels_per_cell=(8,8)).

۳. **ساخت هیستوگرام جهت‌ها:** در هر سلول، جهت گرادیان‌ها در ۹ دسته تقسیم‌بندی شده و یک هیستوگرام ساخته می‌شود (orientations=9).

۴. **نرمال‌سازی بلوکی:** چند سلول کنار هم به عنوان یک بلوک در نظر گرفته می‌شوند (2×2 سلول) و بردارها نرمال‌سازی می‌شوند تا اثر نور و کنتراست کاهش یابد (cells_per_block=(2,2) و block_norm='L2-Hys').

خروجی HOG یک بردار عددی یک‌بعدی است (hog_features) که اطلاعات شکل و جهت لبه‌ها را در تصویر خلاصه می‌کند.

۱.۱۵ مرحله ۴: استخراج هیستوگرام رنگ

```
hist = cv2.calcHist([img], [0,1,2], None, [8,8,8], [0,256,0,256,0,256])
hist = cv2.normalize(hist, hist).flatten()
```

از آنجا که برخی تابلوها ممکن است شکل مشابه ولی رنگ متفاوت داشته باشند، علاوه بر HOG از هیستوگرام رنگ نیز استفاده می‌شود. دستور calcHist تعداد پیکسل‌های هر محدوده رنگی را در سه کانال رنگی محاسبه می‌کند. در اینجا برای هر کانال ۸ بازه در نظر گرفته شده است، بنابراین تعداد کل خانه‌های هیستوگرام برابر است با:

$$8 \times 8 \times 8 = 512$$

سپس با normalize، هیستوگرام نرمال‌سازی شده و با flatten به یک بردار یک‌بعدی تبدیل می‌شود.

مرحله ۵: ترکیب ویژگی‌ها

```
feature_vector = np.hstack([hog_features, hist])
```

در این مرحله، دو بردار ویژگی استخراج‌شده (یکی مبتنی بر شکل و لبه، و دیگری مبتنی بر رنگ) کنار هم قرار می‌گیرند و یک بردار ویژگی نهایی ساخته می‌شود. این بردار، نماینده کامل‌تری از تصویر است.

مرحله ۶: ساخت ماتریس ویژگی نهایی

در پایان، بردار ویژگی هر تصویر به لیست features اضافه می‌شود و پس از اتمام حلقه، این لیست به یک آرایه NumPy تبدیل می‌شود:

- هر سطر از X_features مربوط به یک تصویر است.
- هر ستون از X_features یک ویژگی عددی است

بنابراین، خروجی نهایی X_features یک ماتریس دوبعدی است که به عنوان ورودی مدل‌های SVM و Random Forest استفاده می‌شود.

پس از استخراج ویژگی ها ، داده‌ها وارد مرحله‌ی آموزش مدل SVM می‌شوند. از آنجا که SVM به مقیاس ویژگی‌ها حساس است، ابتدا ویژگی‌ها استانداردسازی شده و سپس مدل آموزش داده می‌شود. در نهایت، عملکرد مدل با معیارهای مختلف ارزیابی می‌گردد.

استانداردسازی ویژگی‌ها

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

پس از استخراج ویژگی‌ها، هر نمونه یک بردار عددی با ابعاد زیاد است. در این حالت، برخی ویژگی‌ها ممکن است مقادیر بزرگ‌تری نسبت به سایر ویژگی‌ها داشته باشند که ویژگی‌های بزرگ‌تر می‌توانند در تصمیم‌گیری مدل غالب شوند و باعث کاهش دقت شوند.
StandardScaler ویژگی‌ها را به گونه‌ای تبدیل می‌کند که:

- میانگین هر ویژگی تقریباً صفر شود.
- واریانس هر ویژگی تقریباً یک شود.

این کار با تبدیل زیر انجام می‌شود:

$$x' = \frac{x - \mu}{\sigma}$$

که در آن μ میانگین و σ انحراف معیار همان ویژگی در داده‌های آموزش است.

۲.۱۵ ساخت و آموزش مدل

```
svm_model = SVC(kernel='linear', C=1.0)
svm_model.fit(X_train_scaled, y_train)
```

در این مرحله، یک مدل SVC با کرنل خطی ساخته می‌شود:

- kernel='linear' یعنی مدل به دنبال مرز تصمیم خطی (ابرصفحه) در فضای ویژگی‌ها است. این انتخاب برای مقایسه با CNN مناسب است زیرا ساده‌تر و سریع‌تر بوده و نقش استخراج ویژگی‌ها (HOG/Histogram) را برجسته می‌کند.
- پارامتر C میزان جریمه خطاها را کنترل می‌کند:
  - اگر C بزرگ باشد، مدل تلاش می‌کند خطاهای آموزشی را کمینه کند (ممکن است بیش‌برازش شود).
  - اگر C کوچک باشد، مدل مرز نرم‌تری می‌سازد و تعمیم‌پذیری بهتر می‌شود.

پیش‌بینی روی داده تست و محاسبه دقت

```
y_pred_svm = svm_model.predict(X_test_scaled)
print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
```

پس از آموزش، مدل روی داده‌های تست اجرا می‌شود و برای هر نمونه یک برچسب کلاس پیش‌بینی می‌کند. سپس Accuracy محاسبه می‌شود:

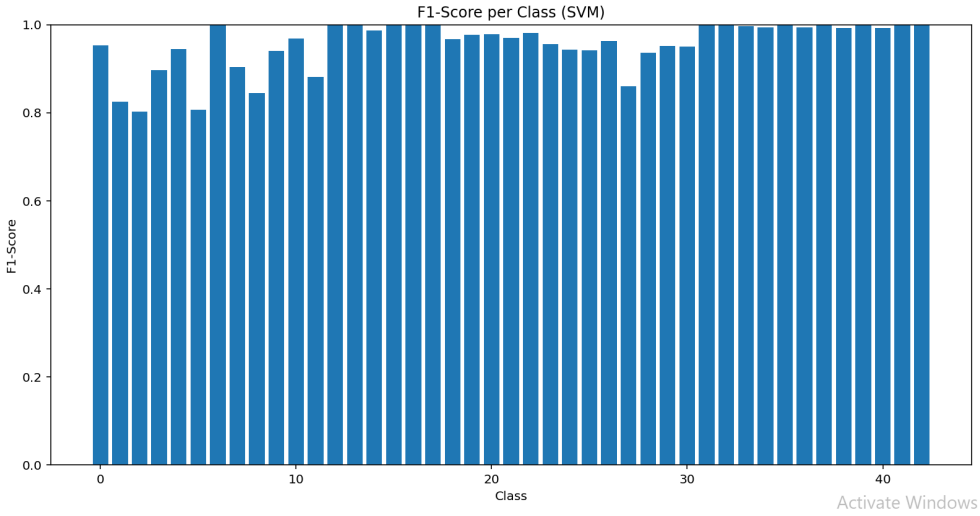
$$Accuracy = \frac{\text{تعداد پیش‌بینی‌های درست}}{\text{تعداد کل نمونه‌ها}}$$

این معیار نشان می‌دهد چند درصد از تصاویر تست به درستی طبقه‌بندی شده‌اند.

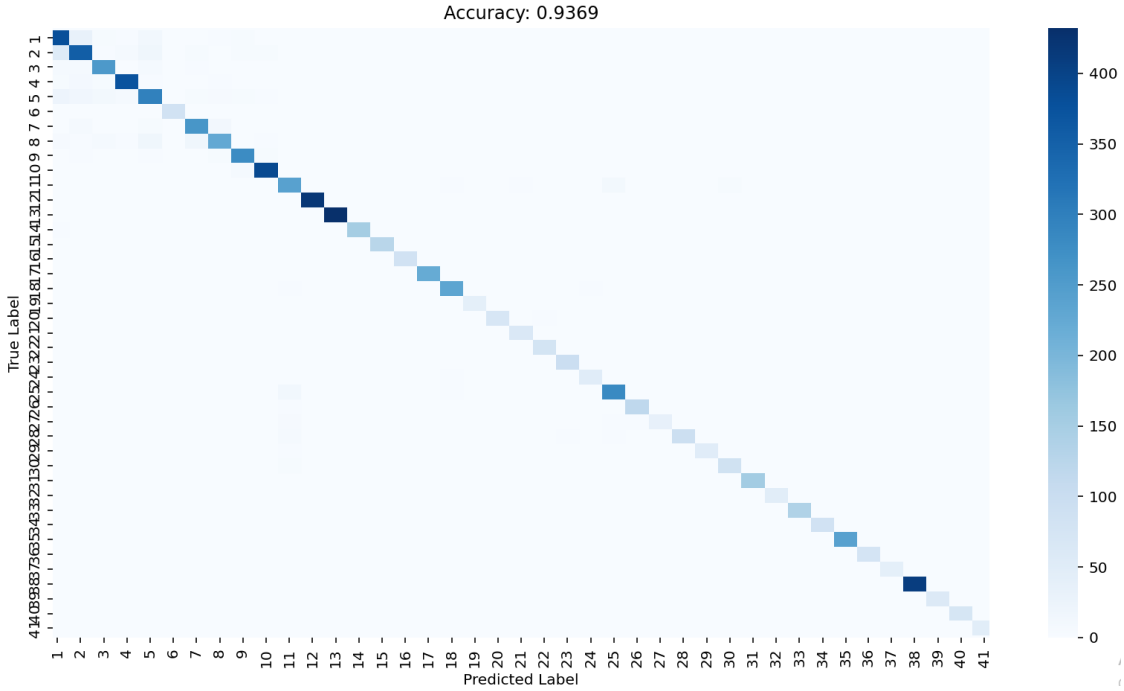
Class	Precision	Recall	F1-score	Support
0	0.93	0.98	0.95	42
1	0.79	0.86	0.82	444
2	0.82	0.79	0.80	450
3	0.89	0.90	0.90	282
4	0.95	0.94	0.94	396
		...		
38	1.00	0.99	0.99	414
39	1.00	1.00	1.00	60
40	0.99	1.00	0.99	72
41	1.00	1.00	1.00	48
42	1.00	1.00	1.00	48
Accuracy = 0.94 (Total samples = 7842)				

Table 2: SVM Classification Report

- **(Accuracy) :** یعنی مدل در مجموع حدود **94%** از تصاویر تست را درست طبقه‌بندی کرده است
- **کلاس‌های ۱ و ۲** نسبت به سایر کلاس‌ها عملکرد ضعیف‌تری دارند (مثلاً F۱ به ترتیب **0.82** و **0.80**). این موضوع می‌تواند ناشی از شباهت ظاهری/رنگی این کلاس‌ها با کلاس‌های نزدیک یا تنوع زیاد نمونه‌ها در همان کلاس باشد.



۳.۱۵ مرحله ۴: ماتریس درهم‌ریختگی (Confusion Matrix)



۱۶ مقایسه روش های SVM و CNN

۱۷ مقایسه دقت کلی

- **CNN**: دقت کلی حدود **0.99**
- **SVM**: دقت کلی برابر **0.94**

اختلاف حدود ۵ درصدی در Accuracy نشان می‌دهد که CNN توانایی بالاتری در تعمیم و تشخیص دقیق کلاس‌ها دارد.

مقایسه عملکرد روی کلاس‌ها

مدل CNN:

- تقریباً در تمام ۴۳ کلاس، Precision و Recall نزدیک به 1 بوده است.
- ماتریس درهم‌ریختگی نشان‌دهنده خطاهای بسیار محدود و پراکنده است.

مدل SVM:

- در بیشتر اکلاس‌ها عملکرد خوبی دارد.
- در برخی کلاس‌های پرتعداد مانند کلاس‌های ۱ و ۲، F۱-score به حدود 0.8 کاهش یافته است.

۱۸ مخزن گیت هاب

Page Github My